Category:

Network

Name:

N1: Man-in-the-flag

Message:

Description:

Cryptography, often described as the art and science of keeping information secure, achieves this by transforming data into an unreadable format using mathematical algorithms and techniques. Its primary aim is to safeguard sensitive information, ensuring its confidentiality during storage, transmission, and access. The beauty of cryptography lies in its versatility: it can encrypt a wide range of data types, from plain text to binary data and even images, employing a variety of methods. One of the most esteemed and widely-used cryptographic techniques is the Advanced Encryption Standard (AES), trusted for its robust security and efficiency. This cryptographic marvel plays a crucial role in safeguarding information in our digital age.

Details:

In a bustling metropolis, Alex, a brilliant cybersecurity analyst at a prestigious firm, stumbled upon a peculiar HTML file while conducting a routine security audit. The file appeared to be just another webpage, but something about it sparked Alex's curiosity.

As the days passed, the enigmatic HTML file began to reveal its secrets, gradually unveiling an intricate puzzle carefully hidden in its code. With each layer of encryption unraveled, Alex delved deeper into the digital labyrinth, driven by the mystery concealed within. Little did Alex know that this innocuous-looking HTML file would lead to an extraordinary journey of decryption, unraveling a message of intrigue, danger, and, ultimately, a revelation that could alter the course of cybersecurity history.

Your task is to join Alex in deciphering the cryptic code and uncovering the message hidden within, as the fate of this digital enigma rests in your hands.

Obtain and decode the flag by examining the HTML file.

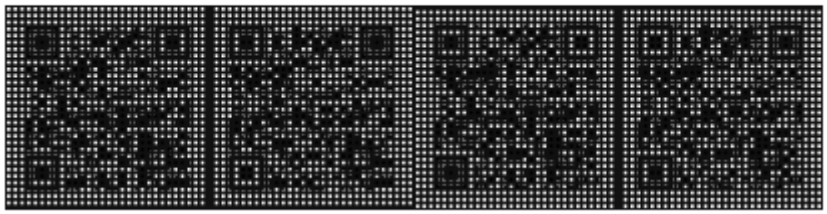
Notes:

You can get the enigmatic HTML file here: n1web.csg2025.local

Instructions:

1. Access the web page : http://n1web.csg2025.local from your attack box.

Go to Download Page



2. The four square images look like QR codes, but they cannot be read as they are.
3. When checking the HTML source, .bg1~4 defined as background in CSS are displayed side by side in a table.

```
.bg1 {background-image: url("
.bg2 {background-image: url("
.bg3 {background-image: url("
.bg4 {background-image: url("
</style>
</head>
<body>
<a href="fetch.php" style="display:block; margin-bottom:1em; font-weight:bold; font-size:1.2em;">Go to Download Page</a><table style="; ba
<tr>
<td><div class="bg1"></div></td>
<td><div class="bg2"></div></td>
<td><div class="bg3"></div></td>
<td><div class="bg4"></div></td>
</tr>
</table>
</body>
</html>
```

4. Let's overlay these images. Below is a sample HTML code for displaying overlaid images:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Overlaying .bg1 to .bg4 with transparency preserved</title>

  <!-- Load existing stylesheet that defines .bg1 to .bg4. Adjust the path as needed -->
  <link rel="stylesheet" href="styles.css" />

  <style>
    /* Minimal utility styles for overlay display (.bg1 to .bg4 are reused from existing CSS) */
    .stack {
      width: min(92vw, 900px);
      aspect-ratio: 16 / 9;     /* Change or remove as needed; alternatively use fixed height */
      position: relative;
      margin: 24px auto;
      background: transparent;   /* Parent container is transparent */
      border: 1px solid #e5e7eb;
      border-radius: 8px;
      overflow: hidden;
    }
    .layer {
      position: absolute;
      inset: 0;

      /* Important: Ensure transparent pixels in PNG show layers underneath */
      background-color: transparent !important;

      /* Reuse background-image from existing .bg1 to .bg4 classes */
      background-repeat: no-repeat;
      background-position: center;

      /* If needed, uncomment to control image sizing */
      /* background-size: contain; */

      pointer-events: none; /* Optional: allow mouse events to pass through */
    }
    /* z-index controls stacking order (higher values appear on top) */
    .z1 { z-index: 1; }
    .z2 { z-index: 2; }
    .z3 { z-index: 3; }
    .z4 { z-index: 4; }
  </style>
</head>
<body>
  <main>
    <h1 style="text-align:center; font-size:clamp(1.1rem, 2.5vw, 1.6rem);">
      Overlaying .bg1 to .bg4 with transparency preserved
    </h1>

    <div class="stack">
      <div class="layer bg1 z1"></div>
      <div class="layer bg2 z2"></div>
      <div class="layer bg3 z3"></div>
      <div class="layer bg4 z4"></div>
    </div>

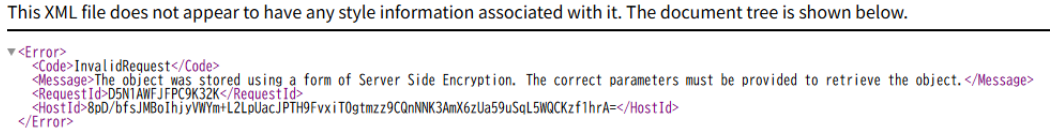
    <!-- Example: To emphasize transparency, add opacity to individual layers
         <div class="layer bg2 z2" style="opacity:.9"></div> -->
  </main>
</body>
</html>
```

Note: You need to create a styles.css file that defines the original .bg1~4.

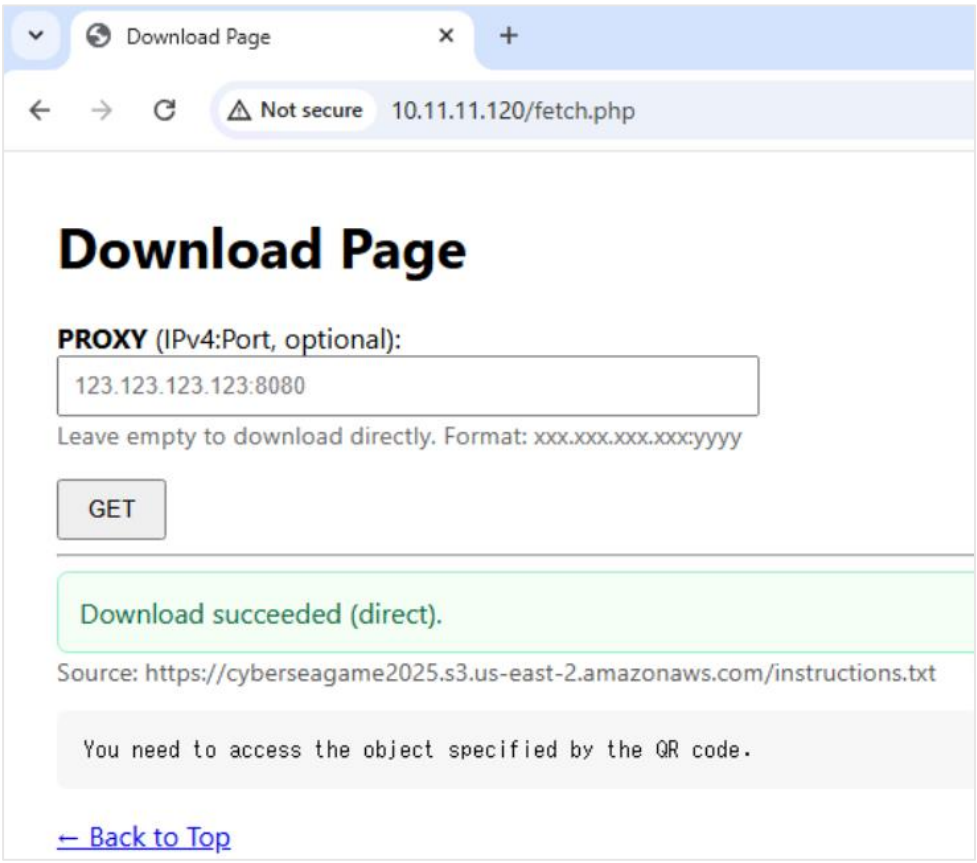
Overlaying .bg1 to .bg4 with transparency preserved



5. The QR code navigates the URL:
   https://cyberseagame2025.s3.us-east-2.amazonaws.com/ltnytc9p84wy8pcw8y4tc.txt
   However, this cannot be viewed because it is encrypted with SSE-C.



6. Return to the web page open in Attackbox and click the “Go to Download Page” link.
7. If you click [GET] without entering anything, a text file will be downloaded from Amazon S3 and its contents will be displayed.



8. However, attempting to access this S3 resource directly yields the same result as the QR code URL.



9. To view content encrypted with SSE-C, you need to include the decryption key in the request header.

   Since decryption is successful when downloading from a web page, you need to build a proxy with TLS inspection enabled to check the request header.

   If you want to use squid, you can use the squid-openssl package.

   You will need to find an available repository or compile from source. Of course, you can also use squid or other proxies compiled for Windows.

   The following is an example of squid.conf with TLS inspection enabled. It is important to configure it to log request headers.

```
http_port 8080 ssl-bump generate-host-certificates=on dynamic_cert_mem_cache_size=16MB tls-
cert=/etc/squid/bumpCA.crt tls-key=/etc/squid/bump.key
cipher=HIGH:MEDIUM:!LOW:!RC4:!SEED:!IDEA:!3DES:!MD5:!EXP:!PSK:!DSS
options=NO_TLSv1,NO_SSLv3,SINGLE_DH_USE,SINGLE_ECDH_USE tls-
dh=prime256v1:/etc/squid/bump_dhparam.pem
sslcrtd_program /usr/lib/squid/security_file_certgen -s /var/lib/squid/ssl_db -M 16MB
sslproxy_cert_error allow all
ssl_bump stare all
http_access allow all
cache deny all
logformat
localtime %{%Y-%m-%d %H:%M:%S}tl.%03tu %6tr %>a %Ss/%03>Hs %<st %rm %ru %[un %Sh/%<a %mt %http::>h
```

10. The following is an example of a squid access log.

```
2025-08-22 02:01:18.252 40 10.11.11.120 NONE_NONE/200 0 CONNECT cyberseagame2025.s3.us-east-
2.amazonaws.com:443 - HIER_DIRECT/52.219.111.10 - Proxy-Connection:%20Keep-
Alive%0D%0AHost:%20cyberseagame2025.s3.us-east-2.amazonaws.com:443%0D%0A
2025-08-22 02:01:18.287 34 10.11.11.120 TCP_MISS/200 736 GET https://cyberseagame2025.s3.us-east-
2.amazonaws.com/instructions.txt - HIER_DIRECT/52.219.111.10 text/plain x-amz-server-side-
encryption-customer-algorithm:%20AES256%0D%0Ax-amz-server-side-encryption-customer-
key:%20NjhBQcOSHGR6eR5bUH9009sTsdvlwo+ZybxG+2rxyhA=%0D%0Ax-amz-server-side-encryption-customer-key-
MD5:%20XJvrCDchW/O49NC2qS3NXA==%0D%0AAccept:%20text/plain%0D%0AHost:%20cyberseagame2025.s3.us-east-
2.amazonaws.com%0D%0A
```

11. The header for decrypting SSE-C is as follows:

```
x-amz-server-side-encryption-customer-algorithm: AES256

x-amz-server-side-encryption-customer-key: NjhBQcOSHGR6eR5bUH9009sTsdvlwo+ZybxG+2rxyhA=

x-amz-server-side-encryption-customer-key-MD5: XJvrCDchW/O49NC2qS3NXA==
```

12. You can capture the flag by the command below:

```
curl -H 'x-amz-server-side-encryption-customer-algorithm: AES256' -H 'x-amz-server-side-encryption-customer-key: NjhBQcOSHGR6eR5bUH9009sTsdvlwo+ZybxG+2rxyhA=' -H 'x-amz-server-side-encryption-customer-key-MD5: XJvrCDchW/O49NC2qS3NXA==' https://cyberseagame2025.s3.us-east-2.amazonaws.com/ltnytc9p84wy8pcw8y4tc.txt
```

